	School of ICT **Sirindhorn International Institute of Technology**
CSS225 Operating System	**Homework 7: Memory-Management Strategies**

Part 1: Multiple-Choice Questions

1. An address loaded into the memory-address register of the memory is _____.
   A) a logical address
   B) a physical address
   C) a virtual address
   D) No correct answer

2. If the base register holds 4501 and the limit register is 5000, then the program can legally access all addresses from 4501 through _____ (inclusive).
   A) 5000
   B) 9500
   C) 9501
   D) 5001

   Refer to the size of this process. As a result, the logical address starts from 0 to 4999.

3. Which one is CORRECT about standard swapping?
   A) It moves only some pages of a process.
   B) It involves moving whole processes between main memory and a disk.
   C) It is popularly used in modern operating systems.
   D) No correct answer.

4. Which one is CORRECT about memory-allocation techniques?
   A) Contiguous memory allocation permits the physical address space of a process to be noncontiguous.
   B) In paging, each process is contained in a single section of memory that is contiguous to the section containing the next process.
   C) Segmentation is a memory-management scheme that supports the programmer view of memory.
   D) No correct answer.

5. Given these five memory partitions of 150KB, 450KB, 75KB, 650KB, and 125KB (in order), how would the best-fit algorithm place a process of size 130KB?
   A) in the 150KB partition
   B) in the 75KB partition
   C) in the 600KB partition
   D) in the 125KB partition

6. Given these five memory partitions of 50KB, 115KB, 210KB, 125KB, and 150KB (in order), how would the first-fit algorithm place a process of size 120KB?
   A) in the 115KB partition
   B) in the 210KB partition
   C) in the 125KB partition
   D) in the 150KB partition

7. Consider the following segment table:

Segment	Base	Length
0	2500	610
1	300	1050
2	7400	1800

What is the physical address of this logical address <1, 345>?
A) 2845
B) 7745
C) 645
D) No correct answer

8. Consider the following segment table:

Segment	Base	Length
0	200	1500
1	5000	500
2	2300	1200

What is the physical address of this logical address <1, 550>?
A) 750
B) 5550
C) 2850
D) No correct answer

9. Which one is INCORRECT about paging?
A) We break the logical memory (logical addresses) of a process into fixed-sized blocks called pages.
B) The page size must be smaller than the frame size.
C) We can avoid external fragmentation.
D) All answers are correct.

10. Given the following page table, which frame number will the page number 3 be stored?

82
5
11
49

A) 82
B) 5
C) 11
D) 49

11. Assuming a 2KB page size and one word equal to one byte, what are the page numbers and offsets for the following address references (provided as decimal numbers): 5500?
A) Page 0, offsets 904
B) Page 1, offsets 1404
C) Page 2, offsets 1404
D) Page 3, offsets 904

12. Consider a logical address space of 16 pages with 1KB page size and one word equal to one byte, mapped onto a physical memory of 512 frames. How many bits are required in the logical address and the physical address?
    A) A logical address needs 13 bits and a physical address needs 15 bits.
    B) A logical address needs 13 bits and a physical address needs 19 bits.
    C) A logical address needs 14 bits and a physical address needs 15 bits.
    D) A logical address needs 14 bits and a physical address needs 19 bits.

13. Consider a logical address space of 32 pages, mapped onto a physical memory of 128 frames with a 2KB frame size and one word equal to one byte. How many bits are required in the logical address and the physical address?
    A) A logical address needs 15 bits and a physical address needs 19 bits.
    B) A logical address needs 16 bits and a physical address needs 18 bits.
    C) A logical address needs 16 bits and a physical address needs 17 bits.
    D) A logical address needs 14 bits and a physical address needs 16 bits.

14. Consider a process of 10500 bytes being allocated into the main memory by paging whose page size is 2KB and one word is equal to one byte. How much is the size of internal fragmentation?
    A) 192
    B) 1500
    C) 1788
    D) 2000

15. Which one is <u>INCORRECT</u> about memory protection in a paged environment?
    A) If a valid-invalid bit is set to valid, the associated page is in the main memory.
    B) If a valid-invalid bit is set to invalid, the associated page is not in the main memory.
    C) If a valid-invalid bit is set to invalid, the associated page might be in the backing store such as a disk.
    D) All answers are correct.

Part 2: Short-Answer Questions

1. (Exercise 8.1) Explain the difference between internal and external fragmentation.

- Internal fragmentation is the area in a region or a page that is not used by the job occupying that region or page. This space is unavailable for use by the system until that job is finished and the page or region is released.
- External fragmentation is unused space between allocated regions of memory. Typically external fragmentation results in memory regions that are too small to satisfy a memory request, but if we were to combine all the regions of external fragmentation, we would have enough memory to satisfy a memory request.

2. (Exercise 8.3) Given six memory partitions of 300 KB, 600 KB, 350 KB, 200 KB, 750 KB, and 125 KB (in order), how would the first-fit, best-fit, and worst-fit algorithms place processes of size 115 KB, 500 KB, 358 KB, 200 KB, and 375 KB (in order)? Rank the algorithms in terms of how efficiently they use memory.

   2.a) Given the following partitions (300KB, 600KB, 350KB, 200KB, 750KB, 125KB) at the beginning. Consider the <u>first-fit</u> algorithm.

Given the following partitions (300KB, 600KB, 350KB, 200KB, 750KB, 125KB) at the beginning. Consider the <u>first-fit</u> algorithm:

- First, the 115KB process will be put in the 300KB partition (leave the free space in this partition to 300KB-115KB=185KB). New partitions are
  (185KB, 600KB, 350KB, 200KB, 750KB, 125KB)
- Next, the 500KB process will be put in the 600KB partition (leave the free space in this partition to 600KB-500KB=100KB). New partitions are
  (185KB, 100KB, 350KB, 200KB, 750KB, 125KB)
- Next, the 358KB process will be put in the 750KB partition (leave the free space in this partition to 750KB-358KB=392KB). New partitions are
  (185KB, 100KB, 350KB, 200KB, 392KB, 125KB)
- Next, the 200KB process will be put in the 350KB partition (leave the free space in this partition to 350KB-200KB=150KB). New partitions are
  (185KB, 100KB, 150KB, 200KB, 392KB, 125KB)
- Next, the 375KB process will be put in the 392KB partition (leave the free space in this partition to 392KB-375KB=17KB). New partitions are
  (185KB, 100KB, 150KB, 200KB, 17KB, 125KB)

By suing the first-fit algorithm, all processes can be allocated into the memory.

2.b) Given the following partitions (300KB, 600KB, 350KB, 200KB, 750KB, 125KB) at the beginning. Consider the <u>best-fit</u> algorithm.

- First, the 115KB process will be put in the 125KB partition (leave the free space in this partition to 125KB-115KB=10KB). New partitions are
  (300KB, 600KB, 350KB, 200KB, 750KB, 10KB)
- Next, the 500KB process will be put in the 600KB partition (leave the free space in this partition to 600KB-500KB=100KB). New partitions are
  (300KB, 100KB, 350KB, 200KB, 750KB, 10KB)
- Next, the 358KB process will be put in the 750KB partition (leave the free space in this partition to 750KB-358KB=392KB). New partitions are
  (300KB, 100KB, 350KB, 200KB, 392KB, 10KB)
- Next, the 200KB process will be put in the 200KB partition (leave the free space in this partition to 200KB-200KB=0KB). New partitions are
  (300KB, 100KB, 350KB, 392KB, 10KB)
- Next, the 375KB process will be put in the 392KB partition (leave the free space in this partition to 392KB-375KB=17KB). New partitions are
  (300KB, 100KB, 350KB, 17KB, 10KB)

By suing the best-fit algorithm, all processes can be allocated into the memory.

2.c) Given the following partitions (300KB, 600KB, 350KB, 200KB, 750KB, 125KB) at the beginning. Consider the <u>worst-fit</u> algorithm.

- First, the 115KB process will be put in the 750KB partition (leave the free space in this partition to 750KB-115KB=635KB). New partitions are
  (300KB, 600KB, 350KB, 200KB, 635KB, 125KB)
- Next, the 500KB process will be put in the 635KB partition (leave the free space in this partition to 635KB-500KB=135KB). New partitions are
  (300KB, 600KB, 350KB, 200KB, 135KB, 125KB)
- Next, the 358KB process will be put in the 600KB partition (leave the free space in this partition to 600KB-358KB=242KB). New partitions are
  (300KB, 242KB, 350KB, 200KB, 135KB, 125KB)
- Next, the 200KB process will be put in the 350KB partition (leave the free space in this partition to 350KB-200KB=150KB). New partitions are
  (300KB, 242KB, 150KB, 200KB, 135KB, 125KB)
- Next, we cannot find a partition big enough to store the 375KB process. Then, this process must wait.

The worst-fit algorithm cannot allocate all processes into the memory.

Rank the algorithms in terms of how efficiently they use memory.

Measured by the number of processes allocated to the memory, both first-fit and best-fit algorithms provide the same efficiency for this given example.

3. (Exercise 8.4) Most systems allow a program to allocate more memory to its address space during execution. Allocation of data in the heap segments of programs is an example of such allocated memory. What is required to support dynamic memory allocation in the following schemes?

   a. Contiguous memory allocation

Contiguous-memory allocation: might require relocation of the entire program since there is not enough space for the program to grow its allocated memory space.

   b. Pure segmentation

Pure segmentation: might also require relocation of the segment that needs to be extended since there is not enough space for the segment to grow its allocated memory space.

   c. Pure paging

Pure paging: incremental allocation of new pages is possible in this scheme without requiring relocation of the program's address space.

4. (Exercise 8.6) On a system with paging, a process cannot access memory that it does not own. Why? How could the operating system allow access to other memory? Why should it or should it not?

An address on a paging system is a logical page number and an offset. The physical page is found by searching a table based on the logical page number to produce a physical page number. Because the operating system controls the contents of this table, it can limit a process to accessing only those physical pages allocated to the process. There is no way for a process to refer to a page it does not own because the page will not be in the page table. To allow such access, an operating system simply needs to allow entries for non-process memory to be added to the process's page table. This is useful when two or more processes need to exchange data—they just read and write to the same physical addresses (which may be at varying logical addresses). This makes for very efficient inter-process communication.

5. (Exercise 8,12) (Paging Method) Assuming a 1-KB page size, what are the page numbers and offsets for the following address references (provided as decimal numbers):

   a. 3085

3085/1024 = 3 and 13/1024 This address will be in Page Number 3 at the offset number 13.

   b. 42095

42095/1024 = 41 and 111/1024 This address will be in Page Number 111 at the offset 111.

   c. 215201

215201/1024 = 210 and 161/1024 This address will be in Page Number 210 at the offset 161.

   d. 650000

650000/1024 = 634 and 784/1024 This address will be in Page Number 634 at the offset 784.

   e. 2000001

2000001/1024 = 1953 and 129/1024 This address will be in Page Number 1953 at the offset 129.

6. (Exercise 8.15) (Paging Method) Consider a logical address space of 256 pages with a 4-KB page size, mapped onto a physical memory of 64 frames.

   a. How many bits are required in the logical address?

Since $256 = 2^8$, then, 8 bits are required for identifying the page number. Since 4KB = 2^{12}, then, 12 bits are required for identifying the offset number. As a result, 8+12=20 bits are needed for a logical address.

   b. How many bits are required in the physical address?

Since $64 = 2^6$, then, 6 bits are required for identifying the frame number. Since 4KB = 2^{12}, then, 12 bits are required for identifying the offset number. As a result, 6+12=18 bits are needed for a physical address.

7. (Exercise 8.20) (Segmentation Method) Consider the following segment table (5 segments):

Segment	Base	Length
0	219	600
1	2300	14
2	90	100
3	1327	580
4	1952	96

   What are the physical addresses for the following logical addresses?

   a. <0, 430>

Segment 0 and at offset 430 → Physical address 219 + 430 = 649.

   b. <1, 10>

Segment 1 and at offset 10 → Physical address 2300 + 10 = 2310.

   c. <2, 500>

Illegal reference, trap to operating system since 500 is larger than the allocated space 100.

d. <3, 400>

Segment 3 and at offset 400 → Physical address 1327 + 400 = 1727.

e. <4, 112>

Illegal reference, trap to operating system since 112 is larger than the allocated space 96.

8. Consider memory management by using segmentation method. From the segmentation table below, answer the following questions.

Segment	Limit	Base
1	550	300
2	60	2200
3	150	1800
4	220	750

8.1) Find the physical address of the logical address <4, 205>.

Physical Address = 955 (as a decimal number)

Show your work to get the answer above; otherwise, points will be deducted.

We have the following steps.

a) Look at Segment 4, the offset number 205 is less than Limit (220) → valid offset number.

b) Base of Segment 4 is 750.

c) As a result, the corresponding physical address is 205+750 = 955.

8.2) Find the physical address of the logical address <3, 170>.

Physical Address = Error due to invalid offset number (as a decimal number)

Show your work to get the answer above; otherwise, points will be deducted.

We have the following steps.

a) Look at Segment 3, the offset number 170 is larger than Limit (150) → invalid offset number.

b) This is an error.

9. Consider a paging memory allocation whose page size is 5KB and one word is equal to 8 Bytes. Answer these three sub-questions.

9.1) How many words are there in one page?

The number of words in one page is = 640 words

Show your work to support your answer above; otherwise, points will be deducted.

The number of words in one page = $\frac{5KB}{8} = \frac{5\times1024}{8} = 640$ words

9.2) What are the page number and page offset for this logical address: 33124?

Page number = 51;

Page-offset number = 484

Show your work to support your answer above; otherwise, points will be deducted.

From $\frac{33124}{640} = 51\frac{484}{640}$.

9.3) Consider a process of 41500 Bytes being allocated into the main memory by this paging memory allocation. How much is the size of internal fragmentation (in the unit of Bytes)?

The size of internal fragmentation (in the unit of Bytes) is: 4580 bytes.

Show your work to support your answer above; otherwise, points will be deducted.

We have the following steps.

a) Page size = $5 \times 1024 = 5120$ bytes

The number of pages to store this process = $\left\lceil\frac{41500}{5\times1024}\right\rceil = \lceil 8.11 \rceil = 9$ pages

b) 9 pages can cover 46080 bytes.

c) The internal fragmentation is 46080-41500 = 4580 bytes

10. A logical address space consists of 8 pages and each page consists of 64 words. The physical address space consists of 64 frames. Answer the following three sub-questions.

10.1) How many (minimum) bits are required in the logical address (in the binary-format number)?

Number of Bits = 9 bits

Show your work to get the answer above; otherwise, points will be deducted.

Number of pages = 8 = 2^3 → 3 bits for page numbers.

Number of words per a page = 64 = 2^6 → 6 bits for offset numbers.

As a result, a binary number for a logical address consists of 3+6 = 9 bits.

10.2) How many (minimum) bits are required in the physical address (in the binary-format number)?

Number of Bits = 12 bits

Show your work to get the answer above; otherwise, points will be deducted.

Number of frames = 64 = 2^6 → 6 bits for frame numbers.

Number of words per a page = 64 = 2^6 → 6 bits for offset numbers.

As a result, a binary number for a physical address consists of 6+6 = 12 bits.

10.3) Given the page table shown below. What is the physical address (in the binary-number format) according to this logical address (paging format): <5, 17>?

Page	Frame	Flag
4	12	v
5	6	v
6	49	v
7	35	v

Physical Address = 000110010001 (binary-number format)

Show your work to get the answer above; otherwise, points will be deducted.

We have the following steps.

a) Given the logical address <5, 17>, we have the corresponding physical address <6, 17>.

b) 6 bits to represent the frame number 6 → 000110.

c) 6 bits to represent the offset number 17 → 010001

As a result, by combining them together, we have 000110010001.